Retail Price Discovery on the Open Web

How Bargain Lane uses Firecrawl and TinyFish inside its operator dashboard

Technical brief · 31 August 2026 · Bargain Lane Operator Dashboard · Prepared for external technical review

Bargain Lane is a six-store closeout and thrift retail group operating across Indiana and Michigan. Its operator dashboard runs buy decisions on liquidation inventory. The single hardest input to that decision — what a customer can actually pay for an item today at a real retailer — has no API behind it. It lives on the open web. This brief describes the pipeline built to answer that question, and the specific role each vendor plays in it.

1 The problem

A vendor offers a truckload and sends a spreadsheet. Several hundred lines of warehouse shorthand — `CASCADE AP COMP FRSH 4CT` — each with a unit cost and a column the vendor labels retail. That last number is the trap. It is supplied by the party with an interest in the load looking good, and in this pipeline it is stored explicitly *to be contradicted, never to be used*. An inflated comparison price makes every cost-as-a-fraction-of-retail figure look better than it is, in the one direction that loses money.

So the buy decision requires an independently sourced street price, per line, from a first-party seller — not a marketplace reseller, and not the vendor's own claim. At several hundred lines per manifest, that is not a research task a human can do. It has to be a pipeline.

2 Where it runs

Surface	What it does	Shape of the work
Manifest Scorer	A vendor manifest is uploaded, every line is priced against real street prices, and the load is scored against a published, versioned buy-criteria document.	Batch. Our largest run to date is 331 lines. Deduplicated by identifier and cache-first on a 90-day TTL.
Price Scan	A store manager on the floor scans a barcode or types a product name and gets back what the item is, what it sells for elsewhere, our own average selling price for the category, and a suggested ticket price.	Interactive, single item, latency-sensitive. A manager is standing in an aisle waiting for it.

Both sit on the same lookup core. The stack underneath is a Cloudflare Worker with D1 and KV, fronting a static vanilla-JS client. Claude Sonnet handles two language tasks: expanding warehouse abbreviations into searchable product names, and extracting prices from unstructured page text where no structured data is available.

3 The lookup pipeline

The pipeline is an escalation ladder, ordered cheapest-first. Each rung runs only because the one before it failed to produce a price.

Step	Action	Cost
1 Identify	Resolve the identifier to a brand, product name and size. A barcode is an excellent identity key and a poor price key, so the name is resolved first and the *name* is what gets priced. Identity may come from product databases and regional grocers; pricing may not.	TinyFish Search — free
2 Search	Query the resolved name against a first-party domain allowlist, then re-filter the results by host, because domain scoping is a ranking preference rather than a guarantee.	TinyFish Search — free
3 Parse snippets	Extract candidate prices from the search snippets already in hand. Many lines finish here.	Model call
4 Fetch pages	Only if the snippets yielded no price, or yielded a spread wide enough to be a conflict rather than a price. Up to three non-marketplace product pages, 10s cap.	TinyFish Fetch — free
5 Render	Only if steps 3–4 produced *no price at all*. Firecrawl renders the page and returns structured product data.	**Firecrawl — 1 credit**
6 Decide	Outlier filtering, pack-size and per-unit normalisation, seller checks, in-stock state, and a confidence grade. A line that cannot be priced is flagged as such rather than left blank.	Free

The distinction the ladder turns on is that a lookup which *found nothing* is a different fact from one that *never ran*. Both produce an empty price, and they lead to opposite actions, so every call is recorded.

4 Why TinyFish

TinyFish Search and Fetch carry the volume. Search is rate-limited at 30 requests per minute and Fetch at 150 URLs per minute per key, which is comfortably enough that a 300-line manifest fits inside a single call window at the ~2.6s a search actually takes. Both are free on the current tier, which is what makes a per-line lookup across a whole manifest viable at all.

Two properties beyond price matter:

- **Domain-scoped search.** Queries can be biased toward a first-party retailer allowlist, which is the difference between a shelf price and a marketplace reseller's markup.
- **Search doubles as identity resolution.** Searching a bare barcode returns one uncorroborated result. Searching the resolved product name returns ten, across five retailers, which gives the outlier filter something to work against.

TinyFish also exposes a metered browser-automation endpoint. It is deliberately not called. Lines that would require it are flagged and left unpriced, which is visible, rather than quietly billed.

5 Why Firecrawl

Firecrawl is scoped to exactly the two places the free path measurably dies, and to nothing else.

5.1 JavaScript-walled pages

A plain fetch of certain retailer product pages returns navigation chrome and no content after ten seconds. The page is real and the price is on it; it simply is not in the HTML that arrives. Firecrawl renders it.

5.2 Fetch-hostile sites

Home Depot returns HTTP 403 to a plain fetch. Firecrawl's `proxy: "auto"` retries through enhanced proxies at no credit surcharge. This matters disproportionately because the big-ticket categories — home improvement, appliances, consumer electronics — are both the most fetch-hostile and the ones where a wrong buy costs the most.

5.3 The structured product format — the actual reason

The decisive advantage is not rendering, which several tools do. It is that `formats: ["product"]` returns `price.amount` and `availability.inStock` as structured data. A rendered page therefore never has to pass through a language model to yield a price.

That removes an entire class of failure. Scraped markup routinely contains lone surrogates and control characters, which survive JSON serialisation and are then rejected by the model API as malformed — taking the whole request with it. On one 331-line run this accounted for roughly a third of all price parses failing, and those lines then read as "no first-party price" when the truth was that we never managed to ask. Structured extraction deletes that failure mode and a model call per line along with it.

The request as sent:

```
POST https://api.firecrawl.dev/v2/scrape
{
  url,
  formats:        ["product", "markdown"],
  onlyMainContent: true,
  proxy:          "auto",       // clears the 403-ing retailers
  maxAge:         172800000,    // a two-day-old price is still the price
  timeout:        30000,
  location:       { country: "US", languages: ["en-US"] }
}
```

`maxAge` is set to 48 hours on the view that a two-day-old shelf price is still the shelf price and costs less to serve. `markdown` is requested alongside `product` as a fallback: if the structured block is absent, the markdown still goes to the parser rather than wasting the credit.

6 Cost control and observability

Firecrawl is the only call in this pipeline that costs money. Everything else is free at current tiers. That asymmetry drives four controls:

- **It runs only after the free path has failed to produce a price.** The test is the presence of a price, not the size of the response — see finding 3 below.
- **Hard credit cap per batch.** Ten credits, which keeps a 331-line manifest inside the 1,000/month free tier even if every eligible line escalates.
- **Every call is logged to D1** — provider, target, credits, success, HTTP status and latency — free calls included. This is what makes "this manifest cost nothing" a fact rather than an assumption. A credit is counted whether or not the scrape succeeded, because a failed scrape can still have been billed and a budget that only counts successes is not a budget.
- **A processing lock** prevents two scheduled runs from escalating the same line twice and spending two credits for one answer.

7 Findings from production

Every rule in the pipeline was written after a real lookup returned a wrong number. The ones most relevant to a tooling discussion:

#	Finding	Consequence
1	Searching a bare barcode returned a single result carrying no price. Searching the resolved product name returned ten, from five retailers — same product.	Identity resolution became a mandatory first step. One uncheckable number became a cluster that can be sanity-checked against itself.
2	A manifest written in plain language priced at roughly 80% coverage. One written in warehouse abbreviations priced at roughly 25% — with every search, fetch and parse succeeding technically and finding nothing.	Abbreviation expansion is not an optimisation. Searching warehouse shorthand is a question nobody would ask out loud.
3	Escalation was gated on response length — under 400 characters meant failure. A product page returned 820 characters of navigation furniture and no price, sailed past the test, and the renderer that would have worked never ran.	The gate now tests for a price. Length was never the question.
4	Domain-scoped search returned five results from domains outside the allowlist — and those were the only ones carrying prices in their snippets.	Scoping is treated as a ranking preference. Results are re-filtered by host before use.
5	A discount retailer's search snippets carried $20.35 for an item a national grocer sells at $2.39.	A price that parses cleanly can still be wrong by an order of magnitude. Corroboration across retailers is not optional.
6	A credit budget built with a coercion that yielded NaN compared false against every check, leaving the paid path effectively uncapped. The tell was a null in the response body the whole time.	Found and fixed. It is also the clearest argument for logging every call: the log is what made an invisible failure visible.

8 Where we would expand

Firecrawl is currently rationed for cost reasons rather than fit. Three expansions are already identified:

- **Escalate on low confidence, not only on total failure.** Today a line reaches Firecrawl only when the free path returns nothing at all. A structured `price.amount` is materially more reliable than a model's reading of snippet text, so any line that currently resolves at low confidence is a candidate.
- **Big-ticket categories.** Home improvement, appliances and consumer electronics are the most fetch-hostile sources and carry the highest per-unit stakes. Lines in these categories that need heavier tooling are currently flagged and left unpriced entirely.
- **Price Scan latency.** The interactive path has a manager standing in an aisle. Skipping the free-then-fallback ladder in favour of going straight to structured data would trade credits for seconds — a trade currently unavailable to us.

The constraint on all three is the same: budget headroom on the one paid line in the pipeline. We would welcome a conversation about what expanded usage could look like.

Prepared from the production integration in the Bargain Lane operator dashboard. Buy-criteria thresholds, internal pricing and sales figures are omitted by design; this covers integration architecture only.